

Introduction to Systems and Control (EE123)
Project Report

CONTROL OF MASS-SPRING-DAMPER SYSTEM

Sofia Orieshkina

EE123 project



**Maynooth
University**
National University
of Ireland Maynooth

02/05/2025

**Department of Electronic Engineering
Maynooth University**



ABSTRACT

By the predictions of the theory we will never achieve the desired output with the proportional open-loop control, but while K is increasing (K - controller variable) the output will be closer to the one that we desired of. In PID (Proportional-Integral-Derivative) control, the Proportional control decreases the rise time and the error, but increases the oscillation, the Integral control eliminates the error, but oscillation will increase, and the Derivative control decreases the oscillation.

All of the measurements for the above the above statements were made with using MATLAB and Simulink in it, with the Coefficients of: M =3 kg, B =1 Ns/m, K =2 N/m for the Mass-Spring-Damper System.

$$\frac{1}{Ms^2 + Bs + K} \quad [1]$$

The main key of doing this project was to analyse different controls systems (as open-loop, close-loop etc.), proportional and PID controllers and which affect they and their values will have on our systems.

TABLE OF CONTENTS

ABSTRACT.....	ii
1 INTRODUCTION	1
2 The Mass-Spring-Damper in engineering.....	3
2.1 Definition of the Spring Damper	3
2.2 Mass-Spring-Damping System	3
2.3 Key Mathematical Properties.....	3
3 Modelling.....	5
3.1 Differential equation model	6
3.2 Laplace transfer function model	7
4 Control system	9
4.1 Purpose of control	9
4.2 Step response.....	9
4.3 Open-loop and closed-loop control.....	10
4.4 PID control.....	11
4.5 Zeigler-Nichols PID tuning.....	11
5 Results.....	13
5.1 Simulink model.....	13
5.2 Lab 4 results	13
5.3 PID tuning experiment	14
5.4 Simulink autotuning PID	15
5.5 Tuned PID results.....	16
5.6 Additional results	16
6 Conclusions.....	23
7 References and Bibliography	24
7.1 References.....	24
7.2 Bibliography.....	24
8 Appendices.....	25
8.1 Appendix 1: Laplace Transformational Table	25

1 INTRODUCTION

In Electronic Engineering, the Mass-Spring-Damper(MSD) system is a basic model that is frequently used to investigate dynamic behavior, vibration analysis and control system modeling. In order to govern the dynamics of an MSD system and guarantee stability, responsiveness, and the intended performance, this project focuses on building and putting into place an electrical control system.\

In order to control the displacement, velocity, and damping properties of the system, we investigate various control strategies in this project, including open-loop, close-loop control, and proportional-integral-derivative(PID) control. We use MATLAB (specifically Simulink in this project) to create a close-loop control system that makes real-time stiffness or damping adjustments.

The objectives include:

- The Mass-Spring-Damping System (Figure 1.0)

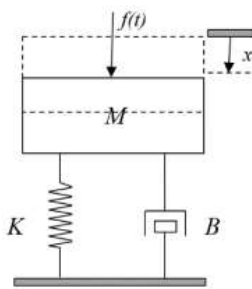


Fig. 1.0 Mass-Spring-Damping System Example

- Modelling (Figure 1.1)

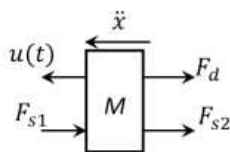


Fig. 1.1 Example for the Free Body Diagram of Mass-Spring-Damping System

- All type of the control systems (Figure 1.2)

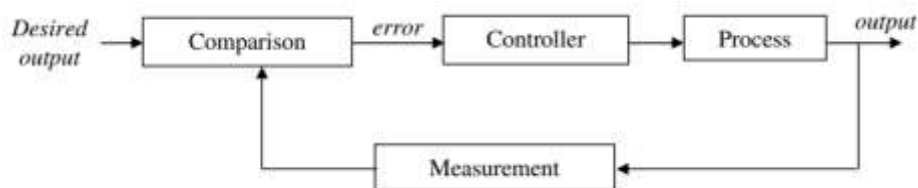


Fig. 1.2 Closed-loop System Diagram

- Results obtained from the MATLAB, using Simulink in it.

This project mainly placed on the open-loop, feedback and Mass-Spring-Damping system, and how they react on the different changes of their variables and the other different factors.

The most important and the basic law that I will use all around my project is “The Second Newton’s Law”. It states: “The net force(F) on a body is equal to the body’s acceleration (a) multiplied by its mass (m)”. The formula of the Law is:

$$F = m * a \quad [1.1]$$

An object accelerates more when there is a greater net force produced on it, and larger mass required more force to accelerate.

2 THE MASS-SPRING-DAMPER IN ENGINEERING

2.1 Definition of the Spring Damper

The device Spring is the elastic device, but generally solid material (usually metal) bend or molded into a shape, particularly a coil, that can regain its shape after being compressed or stretched [1]. Hooke's Law, which states that the force(F_s) a spring pushes back is directly related to the distance(x) from its equilibrium length, is followed by an ideal spring [1]:

$$F_s = k * x \quad [2.1]$$

where k is a constant factor characteristic of the spring.

In physics, damping is the process of restraining of vibratory motion, such as mechanical oscillation, noise, alternating electric currents, by dissipation of energy [2]. The force damping(F_d) with mass(m) and a velocity(v) is:

$$F_d = m * v \quad [2.2]$$

2.2 Mass-Spring-Damping System

As we defined formulas for Spring and Damper, using 2nd Newton's Law (referring on the figure 2.0), we have that:

$$f(t) = F_s - F_d = Ma, \text{ where } F_s = Kx \text{ and } F_d = Bv$$

We can say that $v = \frac{dx}{dt}$, and $a = \frac{d^2x}{dt^2}$, thus:

$$f(t) - Kx - B \frac{dx}{dt} = M \frac{d^2x}{dt^2} \quad [2.3]$$

And using Laplace Transformation:

$$F(s) = K * X(s) + Bs * X(s) + Ms^2 * X(s) \quad [2.4]$$

Hence, Transfer Function is:

$$\frac{X(s)}{F(s)} = \frac{1}{Ms^2 + Bs + K} \quad [2.5]$$

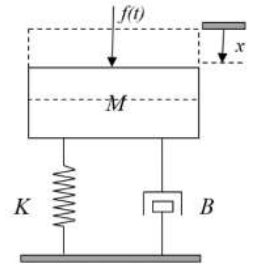


Figure 2.0 Mass-Spring-Damper diagram

2.3 Key Mathematical Properties

Mass-Spring-Damping System is the second order system, which is the lowest order system capable of an oscillatory response to a step unit. The general formula of the second order System.

$$G(s) = \frac{K \omega_n^2}{s^2 + 2\zeta \omega_n s + \omega_n^2} \quad [2.6]$$

Where K is steady-state gain, ω_n is the undamped natural frequency and ζ is the damping ratio. ω_n and ζ are positive real numbers such that $\omega_n \geq 0$ and $\zeta \geq 0$.

The damping ratio determines the location of the poles in the complex s-plane, which directly affects the system's transient response (oscillations, setting time, overshoot).

The diagrams of the second order systems are below:

1. Real equal poles, Critically Damped (Figure 2.1)

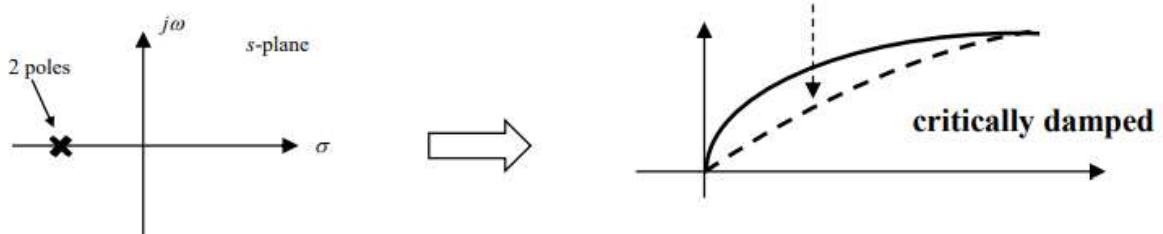


Fig. 2.1 Diagram of real equal poles and graph of critically damped system

2. Real distinct poles, Overdamped (Figure 2.2)

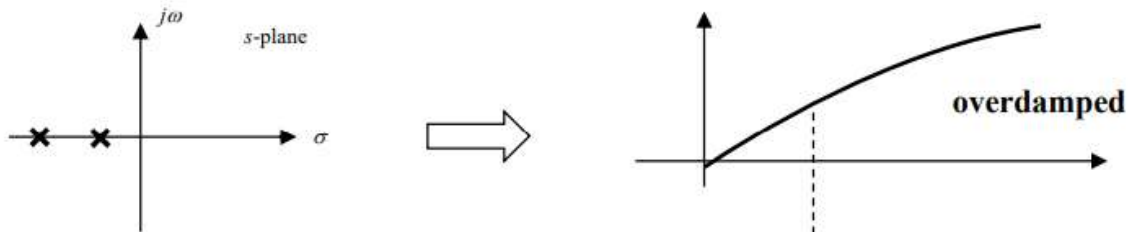


Fig. 2.2 Diagram of real distinct poles and graph of overdamped system

3. Complex poles, Underdamped (Figure 2.3)

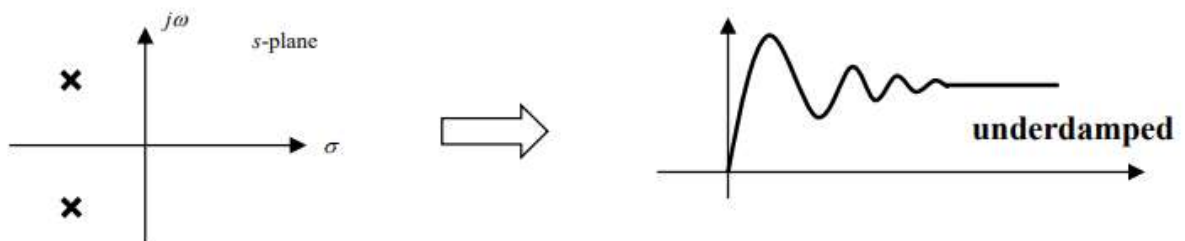


Fig. 2.3 Diagram of complex poles and graph of underdamped system

4. Imaginary Poles, Undamped (Figure 2.4)

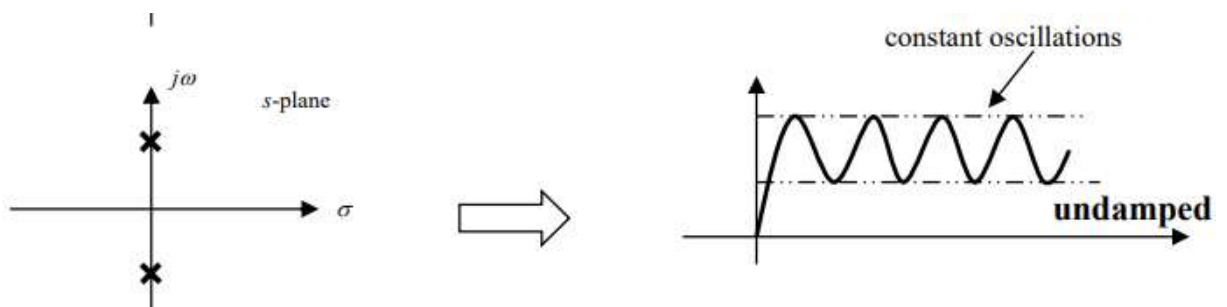


Fig. 2.4 Diagram of imaginary poles and graph of undamped system

3 MODELLING

We must first model the system in order to analyse its response and forecast its performance before we can create a controller for it. Usually, a mathematical description of the system's dynamical properties serves as the foundation for the model. A mathematical model is the term used to describe this mathematical description. Differential equations, for continuous-time systems, or difference equations, for discrete-time systems, are typically used to develop actual mathematical models of various systems.

The first time of the systems is the Static systems. Static system is the system which is always remain unchanged and steady-state over time [3]. For the static system modelling we are going to use Ohm's Law, which states that the voltage(v) across a resistor(R) is directly proportional to the current(i) flowing through it:

$$v = iR \quad [3.1]$$

Also it is the modelling of the dynamic system which can be defined as in the followed diagram (Figure 3.1)

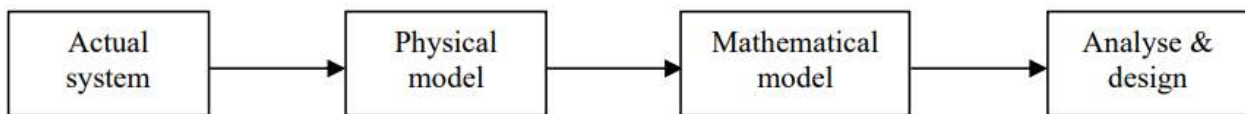


Fig. 3.1 Stages of Dynamic System Investigation

For the physical relationship between each components of the Electrical System we can complete table (Figure 3.2) of relationships.

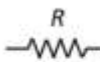
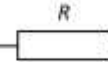
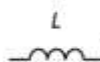
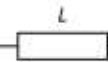
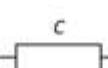
Component	Law	Symbol
Resistance (R)	$v = Ri$	 OR 
Inductance (L)	$v = L \frac{di}{dt}$	 OR 
Capacitance (C)	$i = C \frac{dv}{dt}$	 OR 

Fig. 3.2 Physical Relations for each component

Free Body Diagram is used for the analysing of mechanical systems in the main concepts of mechanics. Only the forces acting on each mass are visible, and each mass is seen as a free entity separate from the rest of the system. The example of the FBD we can see in the Figure 3.3 below

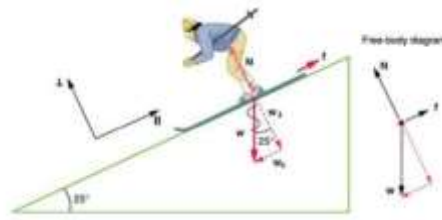


Fig. 3.3 Example of Free Body Diagram

3.1 Differential equation model

In engineering, differential equations are essential since they enable us to model and describe a variety of processes that take place in both technical and natural systems. These formulas show how a function that describes a system's behavior and its derivatives, which show how quickly the function is changing, relate to one another. Thermodynamics, hydrodynamics, electronics, mechanics and another engineering fields all use differential equations because of their universal qualities. Differential equations are used to analyze electrical circuits in electrical engineering. In order to ascertain the properties of systems, Kirchhoff's Laws (Figure 3.4 and Figure 3.5) – which specify the distribution of voltages(V) and currents(I) in a circuit – are converted into differential equations.

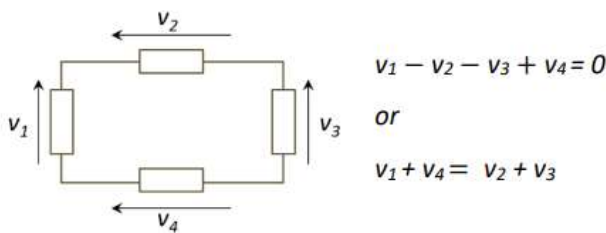


Fig. 3.4 Kirchhoff's Voltage Law (KVL)

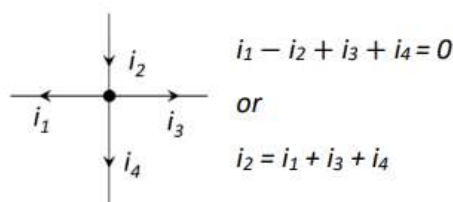


Fig. 3.5 Kirchhoff's Current Law (KCL)

For Differential Equation modelling we are using the knowledge about the Mass-Spring-Damping from the second section from my project. First thing to look at is the Forces, using the Newton's Second Law ("The net force(F) on a body is equal to the body's acceleration (a) multiplied by its mass (m)"). The formula of the Law is:

$$F = m * a \quad [3.2]$$

1. Spring Force(F_s) Opposes displacement $x(t)$, given by Hooke's Law(The Force(F_s) needed to extend or compress a spring by some distance(x) scales linearly with respect to that distance)

:

$$F_s = -kx(t) \quad [3.3]$$

2. Damping Force(F_d) opposes velocity $\dot{x}(t)$, proportional to the damping coefficient:

$$F_d = -c\dot{x}(t) \quad [3.4]$$

3. External Force($F(t)$) applied externally.

The net force of the mass is:

$$F_{net} = F(t) + F_s + F_d = F(t) - kx(t) - c\dot{x}(t) \quad [3.5]$$

The acceleration $a(t) = \ddot{x}(t)$, so:

$$m\ddot{x}(t) = F(t) - kx(t) - c\dot{x}(t) \quad [3.6]$$

Rearranging, we get the second-order linear ordinary differential equation (ODE):

$$m\ddot{x}(t) + c\dot{x}(t) + kx(t) = F(t) \quad [3.7]$$

3.2 Laplace transfer function model

The transfer function is a compact representation of the relationship between the input and an output for a linear system. In this form, we no longer work with ODEs but rather with algebraic expressions. The Laplace Transformation (Figure 3.6) takes a time function, $f(t)$, and transforms it to a Laplace function $F(s)$



Fig. 3.6 Laplace Transformation

$$L[f(t)] = F(s) \quad [3.8]$$

The formula 3.8 is the notations of the Laplace Transformations. L is the Laplace transformation itself, $f(t)$ is the function, s is the rate of change and $F(s)$ is a new transformed function. For the Laplace Transformation it is a specific formula:

$$F(s) = \int_0^{\infty} f(t)e^{-st} dt \quad [3.9]$$

However, in general for the Engineering questions we are using the Laplace Transformational Table, which can be found in the section 8, Appendix 1 of my project.

For Inverse of the Laplace Transform we can use Partial Fraction Method. We can look at this method by looking at an example (Example 3.1) below.

Consider the following Laplace transform:

$$F(s) = \frac{s+2}{(s+3)(s+4)} \quad [3.10]$$

We can perform it as the Partial Transformation:

$$\frac{A}{s+3} + \frac{B}{s+4} = \frac{A(s+4)+B(s+3)}{(s+3)(s+4)} \quad [3.11]$$

Such that we can compare the numerators: $s + 2 = A(s + 4) + B(s + 3)$ [3.12]

By setting $s=-4$ and $s=-3$ we can find that $A = -1$ and $B = 2$.

Now we will use the Laplace Transform Table to identify $F(s)$

$$F(s) = \frac{-1}{s+3} + \frac{2}{s+4} = -1 * \frac{1}{s+3} + 2 * \frac{1}{s+4} \quad [3.13]$$

Let's look in the Table (Figure 3.7)

Exponent	e^{at}	$\frac{1}{s-a}$
----------	----------	-----------------

Fig. 3.7 Part of the Laplace Transfer Table

By taking the value in the table we can now write the $f(t)$

$$f(t) = -e^{-3t} + 2e^{-4t} \quad [3.14]$$

Though, by evaluating the information from our example we have that by using the Laplace transform we can go from the form of $F(s)$ and back to the form of $f(t)$

One of the key advantages of Laplace transforms is the transformation of linear differential equations into algebraic equations.

4 CONTROL SYSTEM

A system is an arrangement of parts working together to complete a task. Anything that has inputs and outputs is considered it.

A control system is a system configuration made up of interconnected parts that will provide the intended system response. It modifies a system's input produce the intended result.

Control is hidden consistence of everyday basics. Control Systems are everywhere around us, if we will look more carefully to the home necessary items, i.e. hairdryer, toaster, charger, mobile phone, we will see that every of them are control systems. The main Control Systems Are Open-loop and Close-loop control Systems about which I will go through later in my project.

4.1 Purpose of control

For first the controllers are for controlling the electrical devices and mechanisms, that we need for gaining the desired output. For example, we need to control the Hairdryer to dry the hair, to make sure that the desired output is exactly as we wanted it to be. The next is to keep away from the dangerous situations, as toxic environment or extreme temperatures. The third is to achieve the output with the controller, while you cannot do that, e.g. working with the Nano-details. The last is to meet desired specifications and modifications, which will be useful and interesting for the consumer, e.g. higher inefficiency, lower costs.

4.2 Step response

Recall that the transfer function of the RC circuit is: $\frac{1}{1+sRC}$

As an example we can use the following standard transfer function of the first-order systems in general (Figure 4.1)

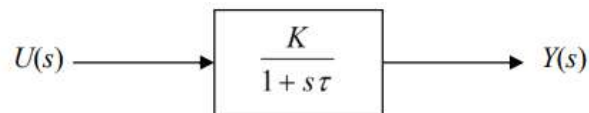


Fig. 4.1 diagram for single tank system

By considering that the step-input has an amplitude of 100 we can use the partial fraction to do the Laplace Transforms as follow:

$$\frac{100K}{s(1+s\tau)} = \frac{A}{s} + \frac{B}{1+s\tau} = \frac{A(1+s\tau)+Bs}{s(1+s\tau)} \quad [4.1]$$

I we will solve it, then we will get:

$$y(t) = 100K(1 - e^{-\frac{t}{\tau}}) \quad [4.2]$$

Graphically the characteristic response for standard first-order system is showed in Figure 4.2 below

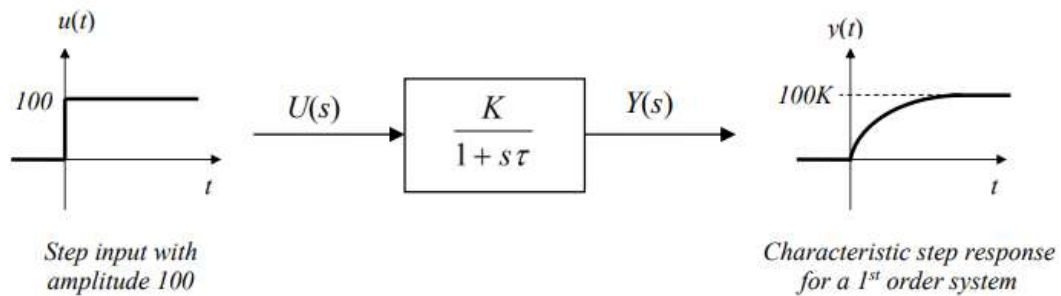


Fig. 4.2 Graphical interpretation

4.3 Open-loop and closed-loop control

The main difference of the Open-loop control system is that it does not have feedback, so there are no measurements to performance. To change the output, the input needs to be adjusted. The simplest example is the toaster with no measurements taken, however adjusting the toasting. The diagram 1.1 below shows the open-loop Control Systems diagram.

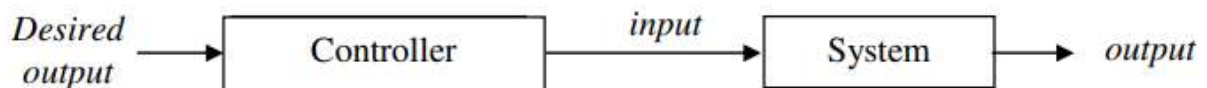


Fig. 4.1 Open-loop control System

The Feedback(Close-loop) Control System takes an input and gives an output with the measurements (if programmed for) to perform. Based on the performance taken it measures the error and adjust its input to get our desired output. The common example of this system is the Home Heating system, which has the simple on/off controller to control the system. The diagram 1.2 represents the basic idea of the Close-loop Control System.

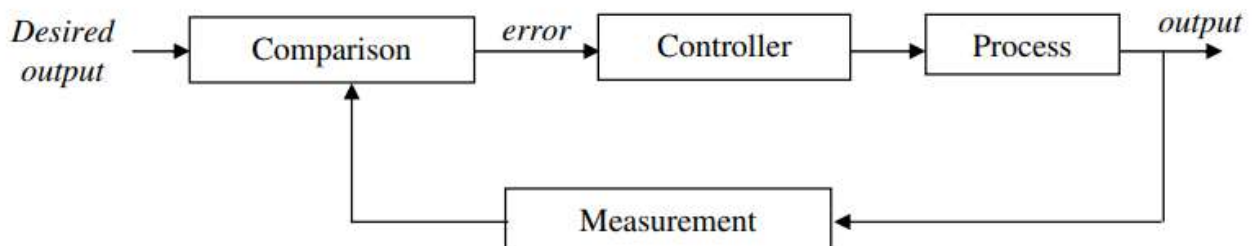


Fig. 4.2 Feedback(Close-loop) Control System

4.4 PID control

The PID controller consists of the sum of three parts, namely proportional, integral and derivative as follows:

$$u(t) = K_p + \frac{K_i}{s} + K_d s = \frac{K_d s^2 + K_p s + K_i}{s} \quad [4.1]$$

Here, K_p , K_i , K_d represent the proportional, integral and derivative gain values respectively.

Integral action allows us to eliminate the steady-state error, i.e. we can zero steady-state error between the desired and actual outputs. Derivative action reduces the system's damping.

The block diagram (Figure 4.3) of the system is:

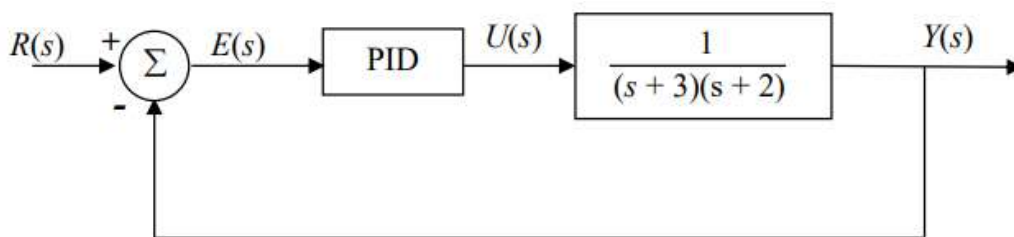


Fig. 4.3 The Block Diagram for PID system

Numerous methods exist that allow us to select the appropriate PID parameters for a whole range of systems. Here, we are only going to consider one of the simpler, commonly used techniques, known as Ziegler-Nichols PID tuning.

4.5 Zeigler-Nichols PID tuning

This is an experiment-based technique that is simple to use. It's based on the step response and does not require a model of the process. Ziegler-Nichols Tuning rules are based on one of two methods. I will look here on the first of the methods called Ultimate Cycle or Stability Limit method and works as follows:

1. Set $K_d = K_i = 0$
2. Increase K_p until the system is marginally stable
3. Note the value of K_p at this point and determine the period of the oscillations. We refer to these values as K_c and t_c respectively
4. Choose control parameters according to the Ziegler-Nichols rules, based on K_c and the period of oscillation t_c as at the Figure 4.4 below

	K_p	t_i	t_d
P	$0.5 K_c$		
PI	$0.45 K_c$	$t_c / 1.2$	
PID	$0.6 K_c$	$t_c / 2$	$t_c / 8$

Fig. 4.4 PID table

This heuristic method will give a reasonably good performance.

The second method is operating upon the shape of the step response of the open-loop system, as follows in Figure 4.5:

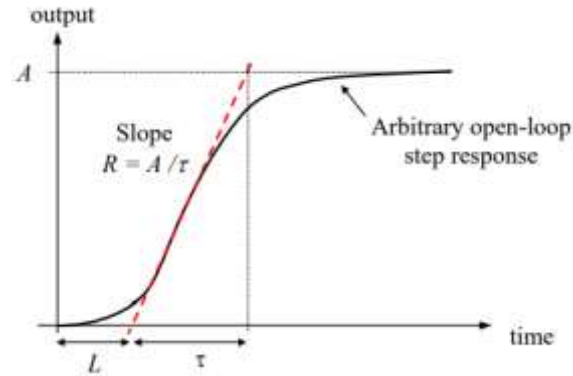


Fig. 4.5 Step response of open-loop system

The controller parameters are calculated based on R (the steepest slope) and L , the deadtime of the system, according to the following table (Figure 4.6):

	K_p	t_i	t_d
P	$1 / RL$		
PI	$0.9 / RL$	$3 L$	
PID	$1.2 / RL$	$2 L$	$0.5 L$

Fig. 4.6 Table of calculated parameters

Only when the system has a well-defined signature and a significant dead-time is this approach effective. A water tank control system is an example of such a system, in which the input flow valve is modified in response to the water tank's height (with a delayed feedback).

5 RESULTS

In this part we will look at the Simulink model from the MATLAB to know and proof new information about Mass-Spring-Damper and PID controllers.

5.1 Simulink model

For my Simulink model for PID tuning (Figure 5.1) I will use such parts: Unit Step, Sum, PID Controller, Transport Delay, Plant $G(s)$, MUX, To Workspace, Scope. For the Plant $G(s) = \frac{1}{Ms^2 + Bs + K} = \frac{1}{3s^2 + s + 2}$ we have that $M = 3$, $B = 1$ and $K = 2$, which we will write to the script in the MATLAB part. As shown in the Figure 5.1 we are using Transport Delay between PID controller and the plant, set its time to 0.01 for doing our Ziegler-Nichols closed loop tuning experiment.

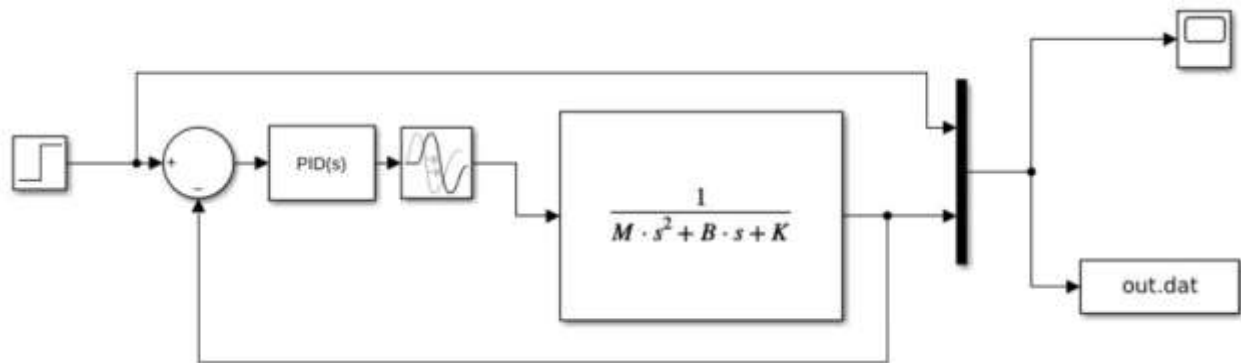


Fig. 5.1 The Simulink Model for PID tuning

For the PID Controller in the Simulink modelling I set the $P = 10$, $I = 0$ and $D = 0$. Stop time is 10.

5.2 Lab 4 results

From the lab 4 Part 1, 2 and 3 we were experimenting with the values and for summary of that I saw that for P-control increasing K_p :

- Reduces the steady-state response, but does not eliminate it
- Increases the speed of the transient response
- Reduces the damping i.e. makes the transient response more oscillatory

We can eliminate steady-state error by adding integral control.

Also, we saw that for PI-control, that any K_i will eliminate steady-state error. Increasing K_i :

- Eliminates steady-state response
- Reduces the damping, i.e. makes the transient response more oscillatory, which is bad

We can increase damping by adding derivative control

We see that derivative control does increase damping (good), but only if our system is noise-free, most real systems are noisy often making derivative control unusable.

5.3 PID tuning experiment

For first results gaining I used the plot(t, y) in the Script of my Simulink Model(Figure 5.2) and it gave me the graph.

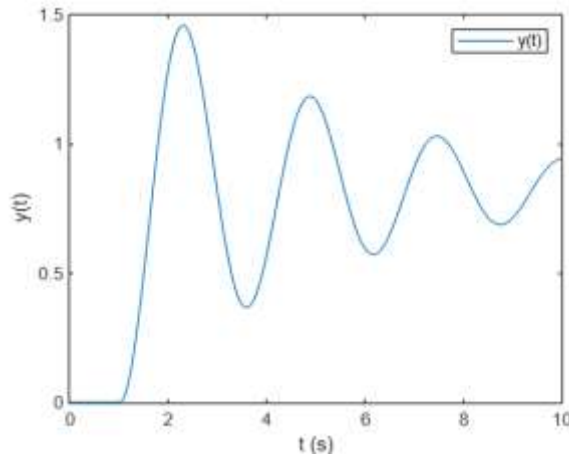


Fig. 5.2 The Graph with $P = 10$

And by changing my P in step of 10, I get that it has perfect sine shape when $P = 100$ (Figure 5.3)

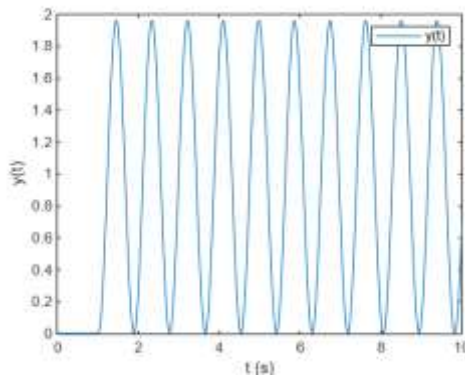


Fig. 5.3 The Graph with $P = 100$

The critical gain of P is $K_c = 100$. For finding t_c I am going to use ginput command in MATLAB. Outputs of ginput are (0.9508, 0) and (1.9525, 0) so the t_c is the distance between the sine waves, the period of the sine and can be calculated as $t_c = 1.9525 - 0.9508 = 1.0017$.

The Ziegler-Nichols PID values are calculated as:

$$K_p = 0.6K_c = 0.6 * 100 = 60 \quad [5.1]$$

$$K_i = \frac{1.2K_c}{t_c} = \frac{1.2*100}{1.0017} \approx 119.7963 \quad [5.2]$$

$$K_d = 0.075K_c t_c = 0.075 * 100 * 1.0017 = 7.51275 \quad [5.3]$$

By putting this Ziegler-Nichols value in my PID block parameters and running the program for 10 s, I get the plotted data on the graph (Figure 5.4).

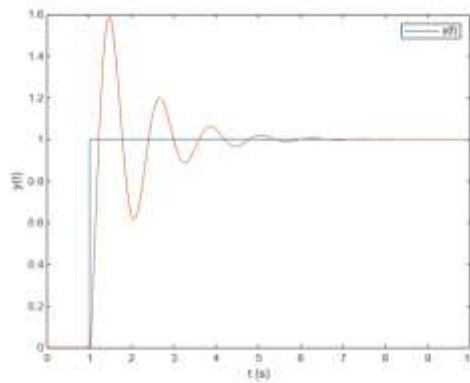


Fig. 5.4 Ziegler-Nichols tuned PID response

5.4 Simulink autotuning PID

For Simulink tuning I will use $P = 1$, $I = 0$, $D = 0$ in my PID controller. In my Simulink I will tune the PID controller. The Response Time is 1 second and the Transient Behaviour is 0.3. This is showing the tuning response as a solid line (Figure 5.5), the dashed line is untuned Simulink model with my P, I and D values.

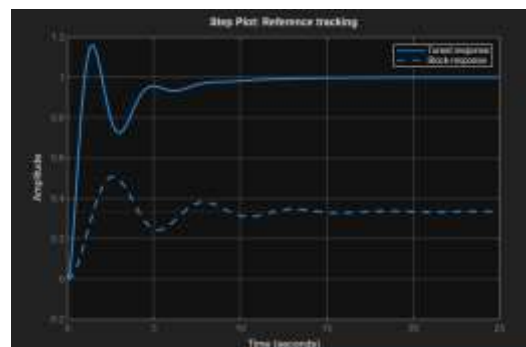


Fig. 5.5 The tuned PID graphs

By clicking on the Show Parameters, it gave me the response measuring (Figure 5.6)



Fig. 5.6 Tuned PID measures

5.5 Tuned PID results

To tune the overshoot of under 10% by slowing my response. While the response time is 0.7 the overshoot is 9.6%, $9.6 < 10$, so we can live it as it is. For settling time, it should be less than 7 seconds, to do so I need to decrease damping, by setting it to 0.62 I get settling time of 6.79 seconds and $6.79 < 7$. In the Figure 5.7 there is the table with those values.

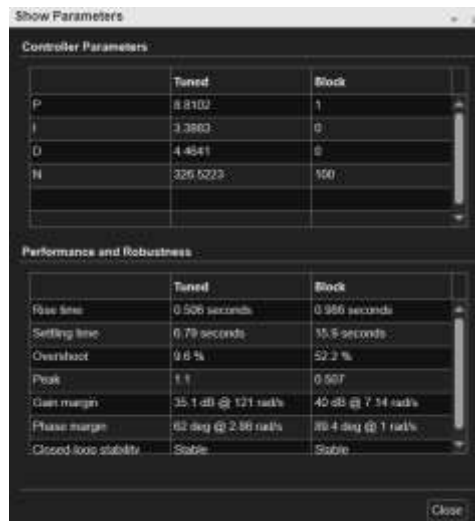


Fig. 5.7 Retuned PID measures

From the part 5.5 we get the tuned PID controller and in this part will be the Results from that tunings. By using the Script of the MATLAB. In Figure 5.8 there is the graph obtained from the tunings.

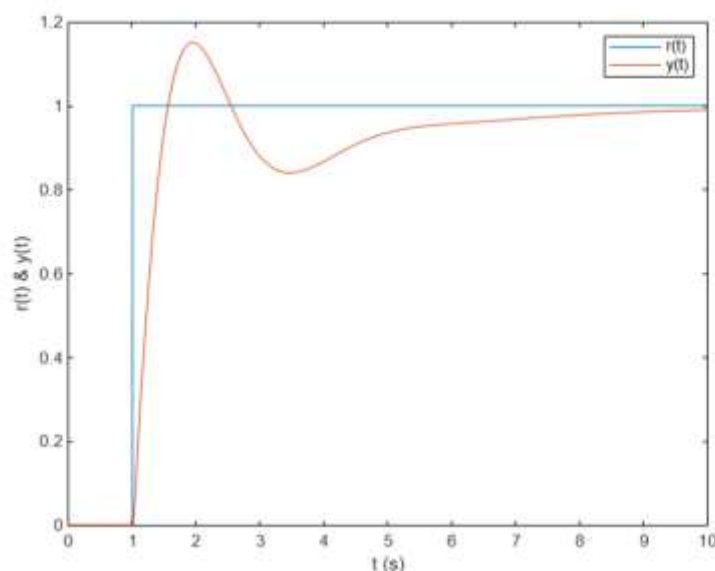


Fig. 5.8 Simulink PID tuner

5.6 Additional results

Another way to design the controller is to do so directly in the MATLAB. To do so I am going to the [scriptfilescript file](#) and creating a 'control design' code section as shown in Figure 5.9.

```
control_design.m x Figure 1 x +
MATLAB Drive/control_design.m
1 %% control design
2 close all
3 s = tf('s');
4 Gs = 1/(M*s^2 + B*s + K);
5 %% open-loop simulation
6 Kp = 1;
7 sys = Kp*Gs;
8 figure; step(sys)
9 title('open-loop control with Kp = 1')
10 %% closed-loop simulation
11 Kp = 10; Ki = 0; Kd = 0;
12 Cs = pid(Kp, Ki, Kd);
13 sys = feedback(Cs*Gs, 1);
14 figure; step(sys)
15 title('closed-loop P control with Kp = 10')
16 %% closed-loop PI control
17 Kp = 5; Ki = 1; Kd = 0;
18 Cs = pid(Kp, Ki, Kd);
19 sys = feedback(Cs*Gs, 1);
20 figure; step(sys)
21 title('closed-loop PI control with Kp = 5, Ki = 1')
22 %% closed-loop PID control
23 Kp = 10.3; Ki = 3.4; Kd = 7;
24 Cs = pid(Kp, Ki, Kd);
25 sys = feedback(Cs*Gs, 1);
26 figure; step(sys)
27 title('closed-loop PID control with Kp = 10.3; Ki = 3.4, Kd = 7')
```

Fig. 5.9 Code for the Control Design

From the part one called 'open-loop simulation' I get the graphs: (Figure 5.10)

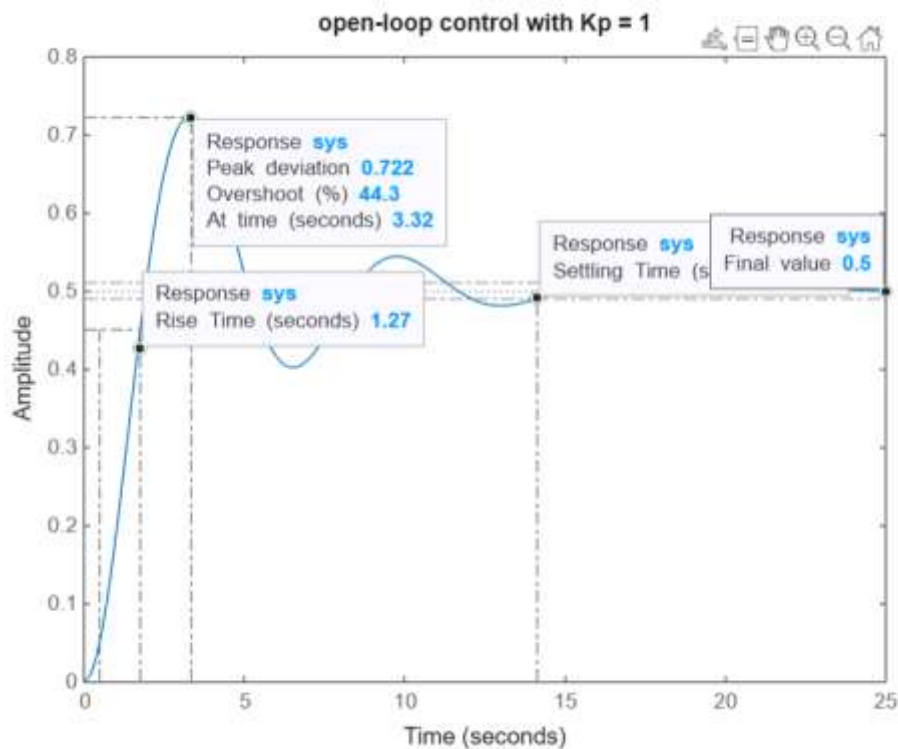


Fig. 5.10 The 1st part graph

From the graph we can inspect the poles by using pole(sys) and the output is

```
ans =
-0.2500 + 0.9682i
-0.2500 - 0.9682i
```

To do the pole-zero plot we are using pzmap(sys). The plot is in the Figure 5.11

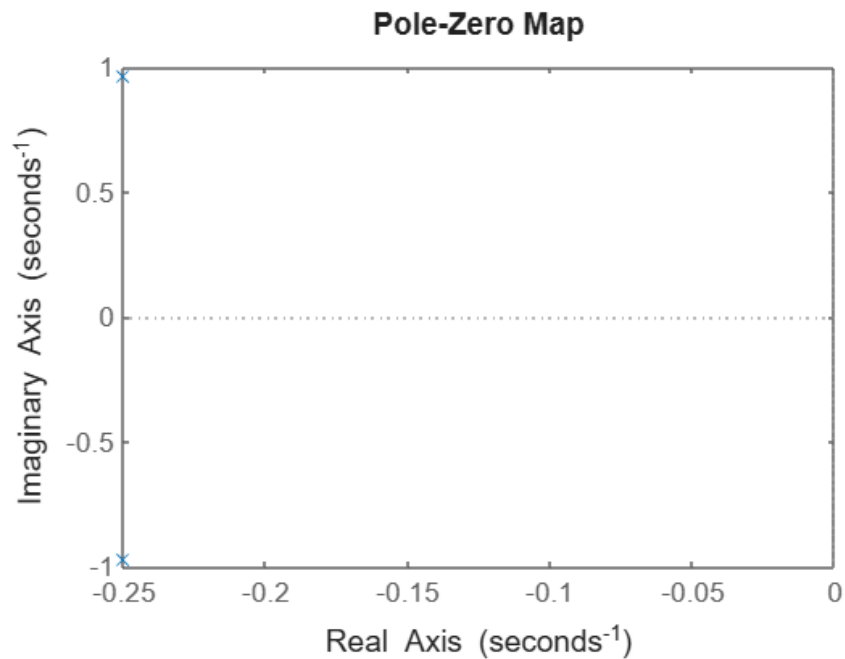


Fig. 5.11 The pole-zero plot

The next step is to do inspect poles, damping ratio and time constant with using damp(sys)

Pole	Damping	Frequency (rad/seconds)	Time Constant (seconds)
$-2.50\text{e-}01 + 9.68\text{e-}01\text{i}$	$2.50\text{e-}01$	$1.00\text{e+}00$	$4.00\text{e+}00$
$-2.50\text{e-}01 - 9.68\text{e-}01\text{i}$	$2.50\text{e-}01$	$1.00\text{e+}00$	$4.00\text{e+}00$

From the second part called ‘closed-loop simulation’ I get the graphs: (Figure 5.12)

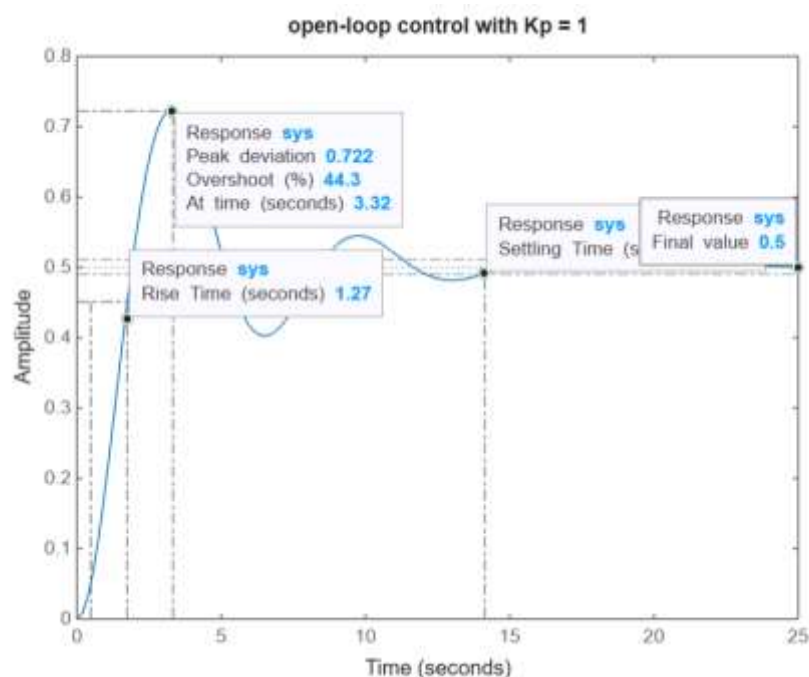


Fig. 5.12 The 2nd part graph

From the graph we can inspect the poles by using `pole(sys)` and the output is

```
ans =  
-0.2500 + 0.9682i  
-0.2500 - 0.9682i
```

To do the pole-zero plot we are using `pzmap(sys)`. The plot is in the Figure 5.13

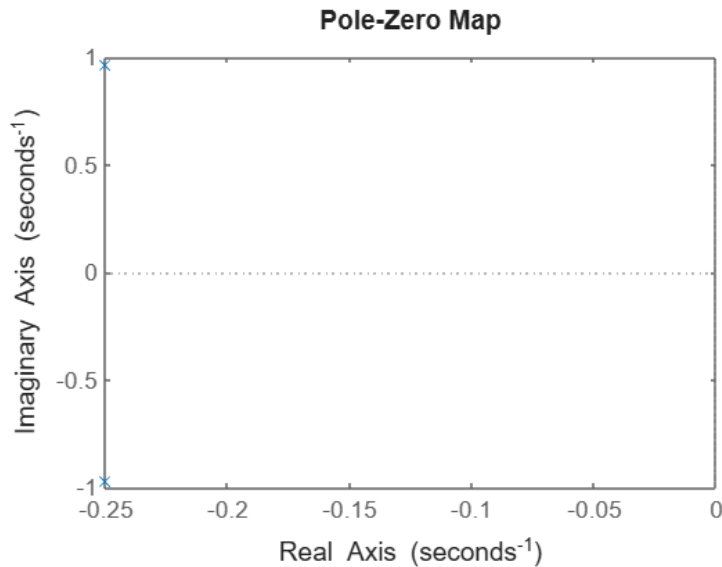


Fig. 5.13 The pole-zero plot

The next step is to do inspect poles, damping ratio and time constant with using `damp(sys)`

Pole	Damping	Frequency (rad/seconds)	Time Constant (seconds)
$-2.50e-01 + 9.68e-01i$	$2.50e-01$	$1.00e+00$	$4.00e+00$
$-2.50e-01 - 9.68e-01i$	$2.50e-01$	$1.00e+00$	$4.00e+00$

From the third part called 'closed-loop simulation' I get the graphs: (Figure 5.14)

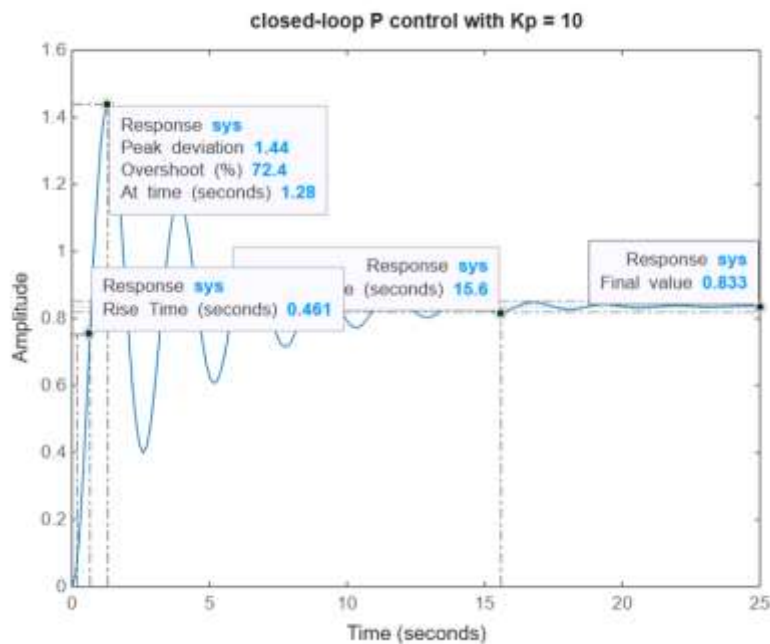


Fig. 5.14 The 3rd part graph

From the graph we can inspect the poles by using `pole(sys)` and the output is

ans =

-0.2500 + 2.4367i

-0.2500 - 2.4367i

To do the pole-zero plot we are using pzmap(sys). The plot is in the Figure 5.15

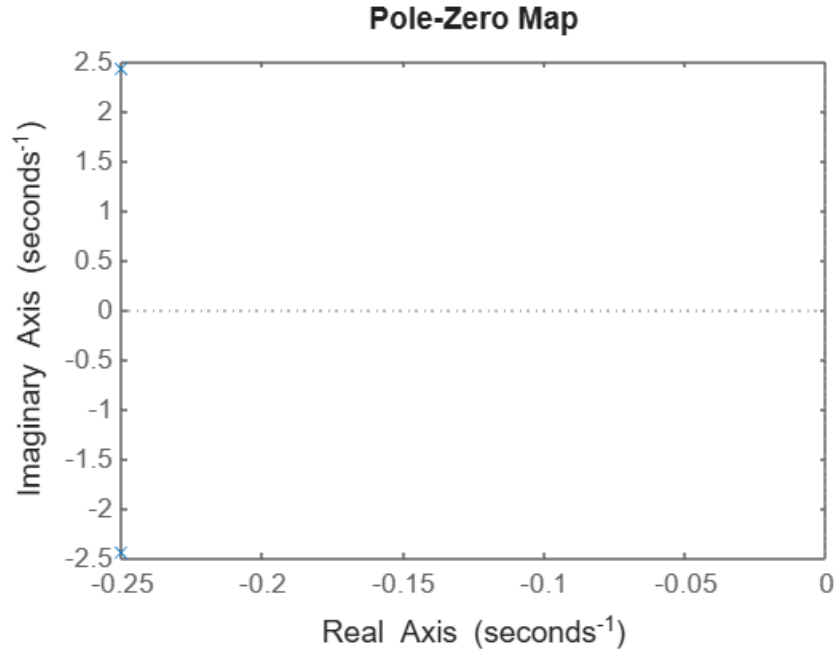


Fig. 5.15 The pole-zero plot

The next step is to do inspect poles, damping ratio and time constant with using damp(sys)

Pole	Damping	Frequency (rad/seconds)	Time Constant (seconds)
-2.50e-01 + 2.44e+00i	1.02e-01	2.45e+00	4.00e+00
-2.50e-01 - 2.44e+00i	1.02e-01	2.45e+00	4.00e+00

From the fourth part called 'closed-loop PI control' I get the graphs: (Figure 5.16)

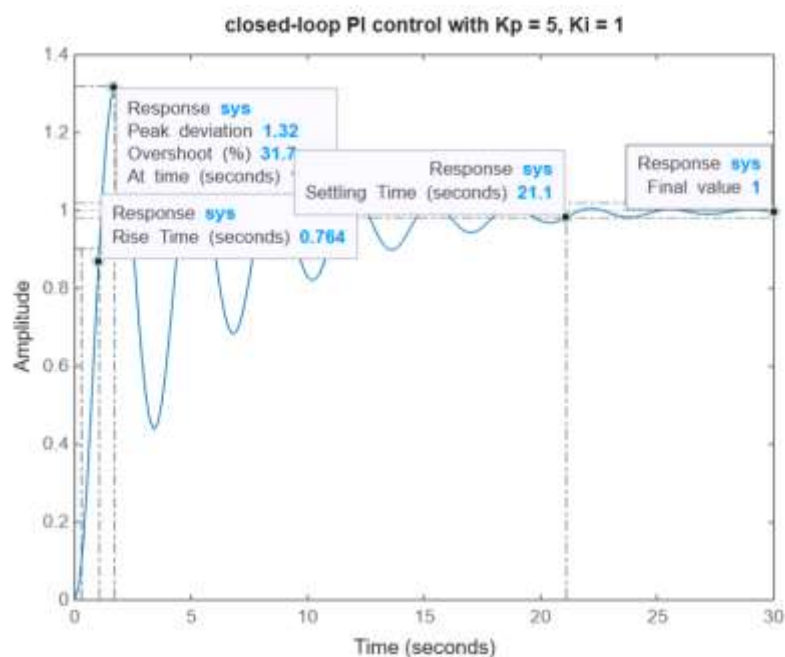


Fig. 5.16 The 4th part graph

From the graph we can inspect the poles by using `pole(sys)` and the output is

ans =

-0.1775 + 1.8485i
-0.1775 - 1.8485i
-0.1450 + 0.0000i

To do the pole-zero plot we are using `pzmap(sys)`. The plot is in the Figure 5.17

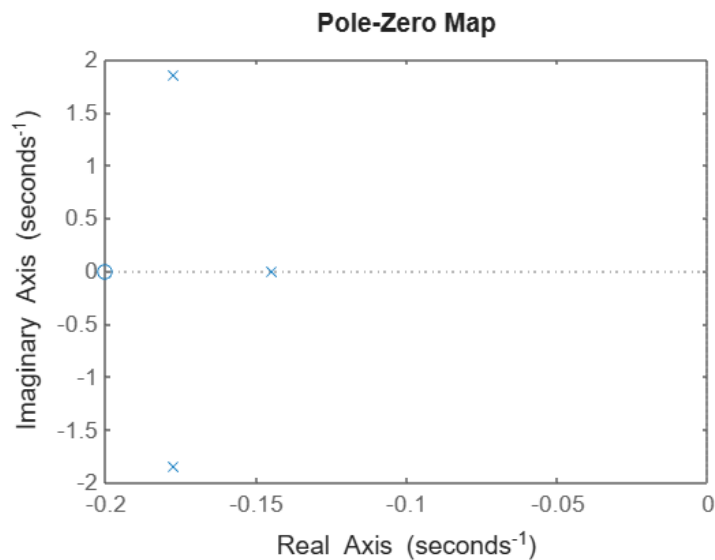


Fig. 5.17 The pole-zero plot

The next step is to do inspect poles, damping ratio and time constant with using `damp(sys)`

Pole	Damping	Frequency (rad/seconds)	Time Constant (seconds)
-1.45e-01	1.00e+00	1.45e-01	6.90e+00
-1.78e-01 + 1.85e+00i	9.56e-02	1.86e+00	5.63e+00
-1.78e-01 - 1.85e+00i	9.56e-02	1.86e+00	5.63e+00

From the last part called 'closed-loop PID control' I get the graphs: (Figure 5.18)

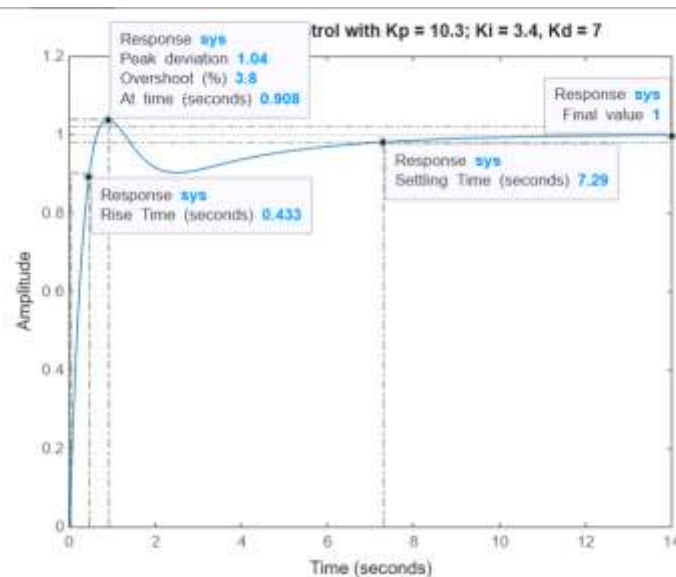


Fig. 5.18 The last part graph

From the graph we can inspect the poles by using `pole(sys)` and the output is

```
ans =  
-1.8257 + 1.2426i  
-1.8257 - 1.2426i  
-0.3486 + 0.0000i
```

To do the pole-zero plot we are using `pzmap(sys)`. The plot is in the Figure 5.19

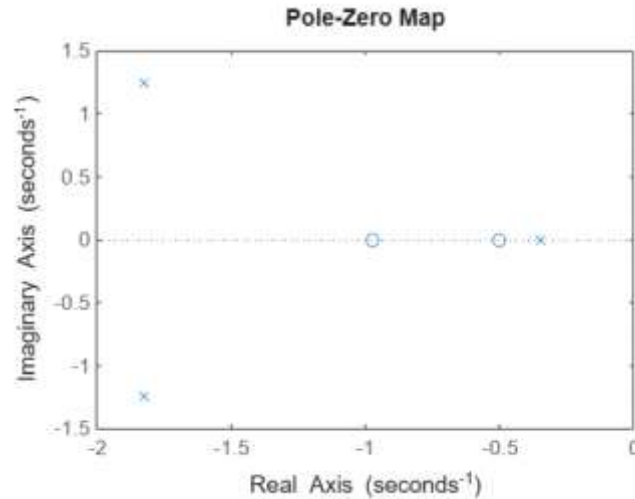


Fig. 5.19 The pole-zero plot

The next step is to do inspect poles, damping ratio and time constant with using `damp(sys)`

Pole	Damping	Frequency (rad/seconds)	Time Constant (seconds)
-3.49e-01	1.00e+00	3.49e-01	2.87e+00
-1.83e+00 + 1.24e+00i	8.27e-01	2.21e+00	5.48e-01
-1.83e+00 - 1.24e+00i	8.27e-01	2.21e+00	5.48e-01

6 CONCLUSIONS

Using MATLAB and Simulink, the “Control of Mass-Spring-Damping System” project effectively investigated the dynamics and control schemes of a second order system. Among the important conclusions are:

1. Closed-loop and Open-loop Control: The project showed that open-loop control lacks feedback, which makes it insufficient for reaching the desired output. On the other hand, by lowering steady-state error and modifying damping properties, closed-loop control, especially with PID tuning, markedly enhanced system performance.
2. Effectiveness of PID Control: It was demonstrated that the Proportional-Integral-derivative (PID) controller was very effective. The derivative term decreased oscillations, the integral term removed steady-state error, and the proportional term decreased rise time and error. The Ziegler-Nichols tuning method produced a stable and responsive system by offering a methodical way to optimize PID parameters.
3. Simulation using MATLAB and Simulink: The simulations demonstrated how different controller gains impact system behaviour and validated theoretical predictions. The project demonstrated how crucial tuning is to meeting requirements like ~~overshoot~~overshoot ($<10\%$) and settling time (<7 seconds).
4. System Modelling: A strong basis for examining the system’s transient and steady-state responses was offered by differential equation and Laplace transfer function models. The system’s performance stability under various control strategies were further confirmed by the pole-zero plots and damping ratios.
5. Practical Applications: From basic mechanical systems to intricate real-world applications like electrical circuit and home heating systems, the project demonstrated the importance of control systems in engineering.

7 REFERENCES AND BIBLIOGRAPHY

7.1 References

1. Wikipedia. Last Updated 21 April 2024. Spring(devices). Available at:
[https://en.wikipedia.org/wiki/Spring_\(device\)](https://en.wikipedia.org/wiki/Spring_(device))
2. Britannica. Last Updated 22 March 2022. Damping. Available at:
<https://www.britannica.com/science/damping>
3. Orapi. Last Updated 2025. Static and Dynamic Systems. Available ~~At~~at:
<https://orapiasia.com/static-and-dynamic-system-difference-and-information/#:~:text=A%20static%20system%20remains%20unchanged,absence%20of%20motion%20or%20change>.

7.2 Bibliography

1. Dr. Seán Doherty, Lecture Notes Sections 1-9, Tutorial Sheets 1-5 | EE114/EE123 Introduction to the Systems and Control.
2. Dr. Séamus McLoone, Lecture Notes Sections 1-9, Tutorial Sheets 1-5 | EE114/EE123 Introduction to the Systems and Control.

Figures from the parts 1 – 4 and Appendix 1 “Laplace Transformational Table” were taken from Lecture Notes. All rights

belong to authors of Lecture Notes for EE114/EE123, Dr. Seán Doherty and Dr. Séamus McLoone.

8 APPENDICES

8.1 Appendix 1: Laplace Transformational Table

Function name	Time domain function $f(t)$	Laplace transform $F(s) = L\{f(t)\}$
Constant	a	$\frac{a}{s}$
Linear	t	$\frac{1}{s^2}$
Power	t^n	$\frac{n!}{s^{n+1}}$
Exponent	e^{at}	$\frac{1}{s-a}$
Sine	$\sin at$	$\frac{a}{s^2 + a^2}$
Cosine	$\cos at$	$\frac{s}{s^2 + a^2}$
Hyperbolic sine	$\sinh at$	$\frac{a}{s^2 - a^2}$
Hyperbolic cosine	$\cosh at$	$\frac{s}{s^2 - a^2}$
Delta function	$\delta(t)$	1
Delayed delta	$\delta(t-a)$	e^{-as}
Derivative	$\frac{df}{dt}$	$sF(s) - f(0)$
Derivative (k^{th})	$\frac{d^k f}{dt^k}$	$s^k F(s) - s^{k-1} f(0) - s^{k-2} f'(0) \dots - f^{(k-1)}(0)$
Integral	$\int_0^t f(\tau) d\tau$	$\frac{F(s)}{s}$

Table of common Laplace Transforms – all functions are defined for $t \geq 0$